Table of Contents

Session 8 - Multi-Agent DAG Orchestration

The Final Architecture

Before we proceed part 1, here's the paper released on 18th May 2026. Its recommendations are very similar to what we are teaching since EAG V1! We seem to be on the right track since long! 😇

Before we proceed part 2. What if the whole agentic orchestration was a graph? What would be possible?

Untill **Session 7** we had a single-loop agent. One Perception call, one Decision call, one Action dispatch per iteration, with vector memory underneath. Three concrete pieces of work that could in principle run at the same time were *serialised* by the iteration rule.

But a query like **"find the populations of London, Paris, and Berlin and tell me which two are closest in size"** cost three sequential fetches because **Decision can only dispatch one tool call per iteration** and **Perception only sees the next fetch's necessity after the previous fetch has been saved in the history**.

Session 8 removes this barrier! The agent loop generalises **from a single iterating role** to **a directed acyclic graph (DAG) of specialised skills**. A Planner generates the **graph**; an **Executor** runs every node whose predecessors are complete; **nodes that share no dependencies run in parallel** under `asyncio.gather` . The same populations query that took **eleven iterations and one hundred twenty-five seconds** under Session 7 takes **seven nodes and sixty-two seconds under Session 8**. The token bill drops from **fifty-four thousand input tokens** to **seventeen thousand**!

The architecture is faster because the work is structured to expose the concurrency/parallelism that was always present in the query.

The session also documents two findings discovered during validation that illustrate where the design draws its lines. The Critic loop, an LLM that reads another LLM's output and verdicts pass-or-fail, is wired correctly and unit-tested for its splice mechanics. The Critic is also, on the forcing queries attempted, a rubber stamp. The wiring is mechanism; the verdict quality is policy; they are separate problems and they are addressed in separate places. Bringing that distinction to surface, is part of what this session teaches.

Where are we right now

By the end of Session 7 the agent had four named roles communicating through Pydantic contracts, vector memory through a FAISS index over `MemoryItem.embedding`, two MCP tools for document indexing and vector search, and a gateway with seven worker providers and a router pool. The eight-query base set passed. Vector retrieval was demonstrably load-bearing on the synonym-recall query; the architectural argument for embeddings rested on that one demonstration.

The architecture was correct for the queries in scope. The architecture also carried a constraint that did not surface in the eight base queries but became visible as soon as the agent was asked to coordinate work across multiple independent sub-tasks. The Session 7 agent loop runs Perception, then Decision, then Action, then memory writes, then loops. Decision emits at most one tool call per iteration. *A query that needs three web fetches against three different URLs cannot, under this loop, issue the second fetch until the first fetch has completed and the result has landed in the next iteration's memory hits and history.* **The serialisation is structural**.

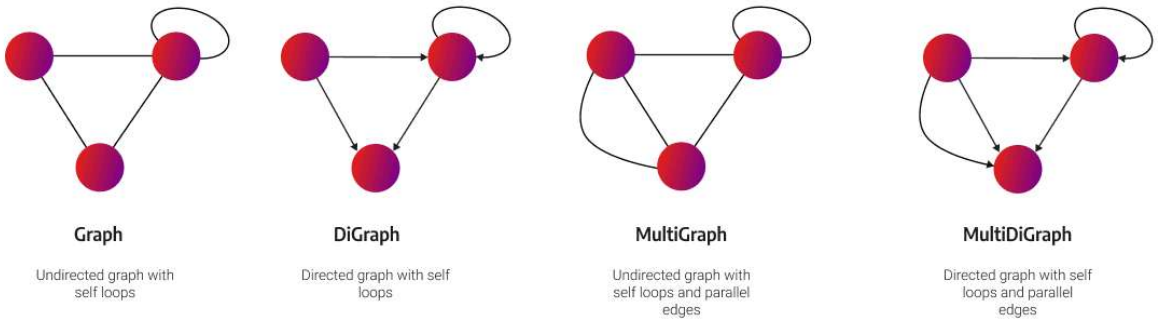
A second cost compounds the first. The Perception and Decision prompts on every iteration include the running history. By iteration ten the prompts are paying for nine previous iterations' worth of tool outputs, memory hits, and goal-list snapshots (**this very difficulty was faced by a lot of students in assignment**). The token bill scales as the iteration count squared in the worst case, and the iteration count is already linear in the number of sub-tasks. A small architectural inefficiency at the level of the loop translates to a substantial bill at the level of the gateway.

Session 8's queries make these costs visible. The populations query is one example. A query that synthesises across three indexed papers is another; the synthesis goal under Session 7 needs the three papers attached to its prompt, and Perception's attach machinery is single-artifact at a time. Whatever shape the query takes, the underlying observation is the same. The Session 7 loop has a structural ceiling. Session 8 changes the shape of the loop itself.

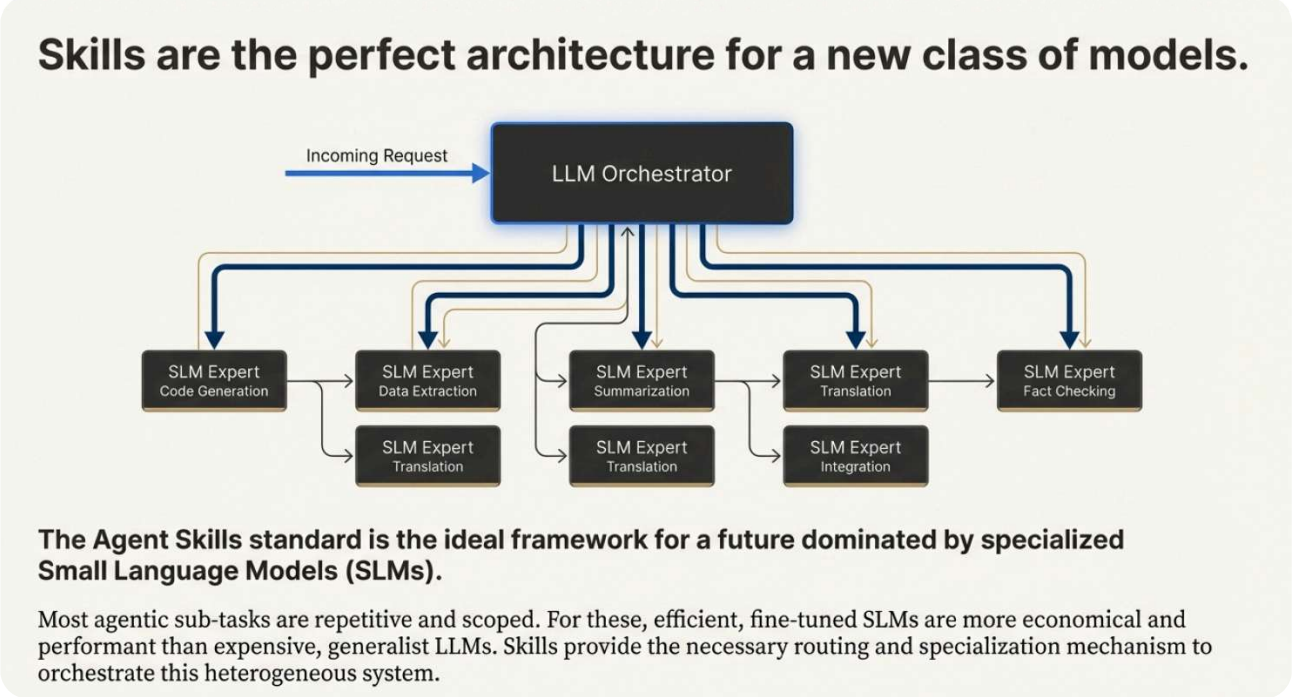
What this session adds

Six changes. Each has its own module; the existing Session 7 code remains byte-identical.

The **first** change is the **orchestrator**. A new file `flow.py` contains a `Graph` wrapper over a NetworkX `DiGraph` and an `Executor` that walks the graph, running every node whose predecessors have completed. Independent nodes run concurrently. The Executor takes a user query and a session id, persists the graph and per-node state to disk, and returns the final answer produced by the terminal Formatter node.



The **second** change is the **skill catalogue**. A new file `agent_config.yaml` enumerates the skills the orchestrator understands: **planner, researcher, retriever, distiller, summariser, critic, formatter, coder, sandbox_executor, and browser**. Each entry names a prompt file, a list of tools the skill may call, a temperature, and a max-tokens budget. A `Skill` class in `skills.py` loads the yaml and renders the skill's prompt against a node's inputs. The skill is not a



The **third** change is the **Planner**. A prompt at `prompts/planner.md`, thirty-nine lines, asks the model to emit the next set of nodes as a JSON object with a `rationale` and a list of `nodes`. Each node names a skill, a list of inputs from the current graph state, and an opaque metadata bag. The Executor reads the planner's output and extends the graph.

```
You are the Planner. Emit the next set of nodes for the orchestrator.

Available skills:
  retriever          search the agent's indexed knowledge base
  researcher         fetch fresh content from the web (URLs, search)
  distiller          extract structured fields from raw text
  summariser         condense long content
  critic             pass/fail evaluation of an upstream node
  formatter          render the final user-facing answer (TERMINAL)
  coder              emit Python (stub; routes to sandbox_executor)
  sandbox_executor   run Python from coder

Output (JSON, no markdown):
{
  "rationale": "<one sentence>",
  "nodes": [
```

```
    ]
}

Reference upstream nodes as "n:<label>" where label matches a
sibling's metadata.label. The final node must be a formatter.

When the user asks to compare or process N concrete items
("compare A, B, C" / "top 3 results"), emit one node per item so
the orchestrator can run them in parallel. Do NOT consolidate.

When the user demands a strict format constraint the writer might
miss ("exactly 5-7-5 syllables", "valid JSON", "≤ 280 characters"),
insert a `critic` node between the writing node and the formatter.
Its input is the writing node id. Its metadata.question repeats
the constraint. If the critic fails, the orchestrator re-plans.

If FAILURE appears in the prompt, do not re-emit the failing step
on the same inputs.

Example:
{"rationale": "Look it up and answer.",
 "nodes": [
   {"skill":"researcher","inputs":["USER_QUERY"],
    "metadata":{"label":"r1","question":"..."}},
   {"skill":"formatter","inputs":["n:r1"],
    "metadata":{"label":"out"}}]
}
```

The **fourth** change is the **Critic**. The skill catalogue marks `distiller` with `critic: true`. The graph extension code, when adding children to a distiller, inserts a critic node on every outgoing edge. The critic's verdict is `pass` or `fail` with a rationale. On `fail`, the orchestrator marks the blocked child as `skipped` and queues one Planner recovery node with the failure rationale. A per-target cap prevents critic-fail loops.

The **fifth** change is the **persistence layer**. A `SessionStore` in `persistence.py` writes the graph as `graph.json` via `nx.node_link_data` and one `NodeState` JSON per node under `nodes/`. All writes are atomic (write-tmp, `os.replace`).

A `SessionLoadError` surfaces when a persisted file fails revival. **Resume** reads the graph back, resets `running` nodes to `pending`, and re-runs from there.

The **sixth** change is the **gateway**. A new gateway, V8, listens on port 8108. V8 adds two columns to every call log (`agent`, `session`), a `/v1/chat/batch` endpoint for **parallel llm dispatch**, a `/v1/cost/by_agent` endpoint that returns token spend grouped by agent label, retry-on-5xx, and an `agent_routing.yaml` that pins specific agent labels to specific providers without consulting the router pool. V7 stays untouched on port 8107.

The total new code is around **fifteen hundred lines** across eight files. The Session 7 modules `perception.py`, `memory.py`, `decision.py`, `action.py`, `artifacts.py`, `vector_index.py`, and `mcp_server.py` are byte-identical to S7. Students with a working Session 7 implementation can understand Session 8 without touching any role from the previous session.

From single-loop to DAG: the mental model

A skill in Session 8 is a triple. A **prompt** file. A **list of MCP tools** the skill is allowed to call. A **temperature**. Nothing else. The skill catalogue is loaded from yaml at startup; the `Skill` class is the data container. There is no per-skill

dispatcher and the same gateway client. What makes one skill different from another is the text in its prompt file.

Critic at 0.0 — it returns {"verdict": "pass" | "fail", "rationale": "..."}. For pass/fail you want the same answer on the same input every time. Any temperature above 0 means the same haiku could get judged differently on Tuesday vs Wednesday, and a downstream re-plan would fire stochastically. Sampling noise on a binary verdict is just noise.

Distiller at 0.1 — extraction ("pull these fields out of this text"). Exploration isn't a virtue here; the field either appears in the input or it doesn't. The unit-test concept "given input X, expect output Y" doesn't work if the extractor is rolling dice.

Researcher at 0.7 — multi-step web search where the model has to decide what query to issue next based on what came back. A bit of exploration is actually useful — at 0.0 the model can get stuck issuing the same search query twice (you saw this happen in the round-1 S7 trace for the populations query).

Planner at 0.4 — decompositions need a little breathing room. At 0.0 you'd get the same DAG shape every time for any given query phrasing, which sounds good until you hit a case where the deterministic decomposition is wrong and there's no second roll of the dice.

NOTE: Gemini 3 documentation recommends keeping temperature at 1.0; the per-skill values above depart from that recommendation deliberately (Critic 0.0 for pass/fail determinism, Distiller 0.1 for extraction). If you observe looping or degraded reasoning on Gemini-routed skills, reset all values to 1.0 and instead constrain behaviour in the prompt.

The **orchestrator** is a DAG or **Directed Acyclic Graph**. **Nodes are** instances of **skills**. **Edges are dependencies**; an edge from node A to node B means B reads A's output. The Executor's loop is short:

```
while there exist incomplete nodes:
    ready = nodes whose every predecessor is complete or skipped
    run all ready nodes concurrently via asyncio.gather
    for each completed node:
        extend the graph with any successors the skill emitted
        persist the graph and the node state to disk
```

The **Planner** is itself a skill. Its prompt asks the model to emit the next set of nodes as a JSON object. The Executor reads the JSON, validates it against the `NodeSpec` Pydantic model, and adds the nodes to the graph. The Planner can be invoked at the start of the run (decomposition) or partway through (recovery from a critic-fail or an upstream-failure node). Its position in the graph is the same as any other skill. The Executor does not special-case the Planner.

This is the meta-level shift that we must absorb. In Session 7, **the model was inside the agent's iteration loop**. The model chose the next tool and the loop ran the tool. In Session 8, **the model writes the graph**. The Planner's JSON output is itself a program that the Executor runs. The same machinery dispatches the Planner and the Researcher and the Critic; the difference between them is what their prompts ask the model to do and what they emit on completion.

The diagram below shows the populations query's full DAG. The Planner fans out into three Researcher nodes which feed a single Coder node; the Coder fans out into a Formatter (which produces the user-facing answer) and a SandboxExecutor (which runs the coder's Python to verify the computation).

Three properties of the DAG are worth naming before going further. **The graph is acyclic by construction**; the planner cannot emit a node that points back at an earlier node. **The graph is dynamic**; new nodes are added as nodes complete, so the structure visible at iteration one is rarely the structure at termination. **The graph is persisted**; every modification of the graph or any node's state writes to disk, so the run can be killed at any moment and resumed cleanly from the last successful write.

The populations query: same query, two architectures

The query is verbatim:

```
Find the populations of London, Paris, Berlin and tell me which two are closest in size.
```

Run against Session 7's agent, the trace shows eleven iterations and a wall-clock of one hundred twenty-five seconds. Iterations one through three handle London; the agent issues a `web_search`, fetches a page, retries on a 4xx, and finally lands content. Iterations five and six handle Paris with one retry. Iteration eight handles Berlin. The remaining iterations are Perception's goal-list

total: twenty-two worker chats, twenty router-classification calls, and eighteen `memory.read` embeddings, one per iteration.

Run against Session 8's flow.py, the trace shows seven nodes and a wall-clock of sixty-two seconds. The gateway log shows fifteen total calls: thirteen worker chats, zero router-classification calls, and two embeddings. The two embeddings are the start-of-run `memory.read` (whose FAISS hits are then rendered into every skill's prompt as the `MEMORY HITS` block — for the populations query the corpus is unrelated, so the block is empty for every skill this run) and the start-of-run `memory.remember`. The zero router calls come from the agent-routing yaml pinning every skill to a specific provider before the request leaves the client.

The per-skill latency table for the Session 8 run reads:

```
node   skill              start (rel)   elapsed   finish (rel)
n:1    planner               0.00 s     2.18 s      2.18 s
n:2    researcher            2.18 s    40.50 s     42.69 s
n:3    researcher            5.80 s    36.89 s     42.69 s
n:4    researcher           10.26 s    32.43 s     42.69 s
n:5    coder                42.69 s    18.56 s     61.26 s
n:6    formatter            61.26 s     1.14 s     62.40 s
n:7    sandbox_executor     62.38 s     0.02 s     62.40 s

wall-clock end-to-end:       62.40 s
sum-of-elapsed (serial):    131.72 s
parallel speedup ratio:       2.11x
```

Three observations are worth making explicit. The three researchers all finish at forty-two point sixty-nine seconds to the millisecond. That is the `asyncio.gather` barrier; the Coder cannot start until every Researcher has landed, so the downstream cost of the parallel layer is the longest branch, not the sum. The speedup ratio is two point one one, not three. The Researchers

layer. The total wall-clock is therefore the serial overhead plus the maximum of the parallel branches; the speedup over a sequential run is the sum of the parallel elapsed times divided by the maximum, attenuated by the serial overhead.

The token bill comparison is structural rather than incidental. **The Session 7 loop re-sends the cumulative history to both Perception and Decision on every iteration**. By iteration eleven **each call carries ten iterations' worth of accumulated memory hits and action results**!! Fifty-four thousand input tokens across the run is the integral.

The Session 8 Researchers, by contrast, are scoped. Each Researcher receives only the user query and its specific sub-question. The Coder receives the three Researcher outputs and the user query. The Formatter receives the Coder's output. No node re-sends history that does not feed into its own decision. The total worker input bill is seventeen thousand tokens, **a factor of 3x lesser** compared to Session 7.

Both runs produce the right answer. Session 7's answer reads: "Berlin and Paris are the two cities closest in size, with a population difference of approximately one point six million." Session 8's answer reads: "The two cities closest in size are Berlin and Paris, with a population difference of one million six hundred forty thousand." The extra precision in the Session 8 answer comes from the sandbox_executor branch. The Coder emits Python code that subtracts the three population values; the formatter quotes the computed integer; the sandbox runs the same code and verifies the answer. The difference is not the model; the difference is that one architecture grounds a precise arithmetic claim in real execution and the other does not.

But the teaching point is not the speedup. The **teaching point is the shape of the trace**. The Session 7 trace under any multi-task query is eleven near-

read when debugging. The trace is what reviewers will read when auditing token spend. The trace is the artifact of the architecture, and the architecture exists in service of producing a trace that can be reasoned about.

Anatomy of one node firing

The **Planner** is the first node to fire on every run. Its inputs are a single token, `USER_QUERY`. Its prompt is the one under `prompts/planner.md`. Its output is a JSON object the orchestrator parses and turns into graph nodes. Walking through the Planner end-to-end on the populations query exposes the full sequence by which an LLM call becomes a graph edge.

The prompt the Planner receives is rendered by `skills.render_prompt` and contains four parts. The first part is the system prompt from `planner.md`, which already lists the available skills with one-line descriptions. The second part is the user query. The third part is the `MEMORY HITS` block — the FAISS-ranked items `memory.read(query)` returns at session start, rendered as one line per hit with the descriptor, the source, and a four-hundred-character preview of either `value.chunk` (for indexed-document chunks) or `value.raw` (for classifier facts). When the query is unrelated to anything in memory the block is empty and the section is omitted; when the corpus has been indexed and the query overlaps with it, the Planner sees what the agent already knows before

no prior nodes, and no tool catalogue. The decomposition is a function of the query and whatever memory has retrieved for it.

The MEMORY HITS plumbing is what carries Session 7's vector retrieval forward into Session 8. The orchestrator calls `memory.read(query)` once at session start, captures the FAISS hits in a local list, and threads that same list through `_run_one` and `run_skill` so every skill's prompt includes the block. No skill's Python code changes; the injection happens inside `render_prompt`. The contract the agent maintains is the one from Session 7's Perception layer — every cognitive role sees what memory contains — applied here to the broader set of skills. `planner.md` carries a corresponding rule: when MEMORY HITS appear, prefer routing through `retriever` or straight to `formatter` rather than scheduling a `researcher` to re-fetch material the agent has already indexed.

The Planner returns:

```
{
  "rationale": "Three populations, then pairwise comparison.",
  "nodes": [
    {"skill": "researcher", "inputs": ["USER_QUERY"],
     "metadata": {"label": "london", "question": "What is the current populatior
    {"skill": "researcher", "inputs": ["USER_QUERY"],
     "metadata": {"label": "paris", "question": "What is the current population
    {"skill": "researcher", "inputs": ["USER_QUERY"],
     "metadata": {"label": "berlin", "question": "What is the current populatior
    {"skill": "coder", "inputs": ["n:london", "n:paris", "n:berlin"],
     "metadata": {"label": "compare"}},
    {"skill": "formatter", "inputs": ["n:compare"], "metadata": {"label": "out"}
  ]
}
```

The **Executor** reads this and calls `Graph.extend_from`. The call validates each node against the `NodeSpec` Pydantic model, resolves the `n:<label>`

newly added node, and writes the resulting nodes and edges into the underlying NetworkX `DiGraph`. The `Skill` for `distiller` has `critic: true`; if the Planner had emitted a distiller, the orchestrator would have inserted a critic node on every outgoing edge from the distiller during this same `extend_from` call. The populations query does not need a distiller, so no critic is inserted.

After `extend_from` returns, the graph contains six nodes: the Planner itself plus the five new nodes. The Planner's status is `complete`. The five new nodes have status `pending`. The Executor calls `ready_nodes`, which returns the three Researcher nodes whose only predecessor is the Planner. Those three nodes are dispatched concurrently. The Coder waits because two of its three predecessors are still pending. The Formatter waits because its predecessor is still pending.

The dispatch path for each Researcher node is identical to the dispatch path for the Planner. `skills.render_prompt` builds the prompt from the Researcher's prompt file plus the resolved inputs (the user query and the question from the metadata bag), plus the same MEMORY HITS block the Planner saw, since both come from the single `memory.read(query)` the executor performed at session start. The gateway client is called with the rendered prompt, the skill's temperature, the skill's max-tokens budget, the agent label (`researcher`), and the session id (`s8-0be05ca6`). The gateway logs the call with both labels. The agent-routing yaml maps `researcher` to the Gemini worker; the request is dispatched directly without consulting the router pool. The Researcher's prompt instructs it to issue one `web_search` and then fetch the top results; an internal tool-use loop handles the multi-turn dance between the model's tool requests and the MCP server's tool results. The loop terminates when the model emits a final text answer instead of another tool call.

The point of walking through one node in detail is that every other node fires

case for the Researcher. The Coder runs through the same path. The Formatter runs through the same path. What varies between them is the prompt file and the tools allowed; the dispatch is identical.

Parallel fan-out and the barrier

The three Researchers in the populations run start at slightly different times (zero, three point six, eight point one seconds after the Planner completes) because `asyncio.gather` schedules them on the event loop in submission order and the gateway client serialises the initial socket writes through a single connection. The variance is in microseconds at the call site and amounts to seconds only because each Researcher's tool-use loop has its own multi-turn interaction with the gateway. The variance is harmless. What matters is that all three finish at forty-two point sixty-nine seconds to the millisecond. The Coder cannot start until every input has landed, and `asyncio.gather` does not resolve until every coroutine has returned. The barrier is the gather's semantics.

The architectural **consequence** is that the cost of a parallel layer is the maximum elapsed time of its branches, not the sum. The populations query has three branches with elapsed times of forty point five, thirty-six point nine, and thirty-two point four seconds. A sequential run would cost the sum, which is one hundred ten seconds. The parallel run costs the maximum, which is forty

A **second consequence** is that the serial overhead does not benefit from parallelism. The Planner (two point two seconds), the Coder (eighteen point six seconds), and the Formatter (one point one seconds) run before and after the parallel layer. Their elapsed time is paid sequentially regardless of how many Researchers are in the parallel layer. As the parallel layer grows, the speedup approaches the asymptotic limit set by the serial overhead. Five researchers would not give five times the speedup; the speedup curve flattens as soon as the parallel layer's maximum elapsed time stops being the dominant term.

A **third consequence** is that the slowest branch dictates the wall-clock. Optimising the average branch is wasted work. Optimising the slowest branch (forty point five seconds in the populations run, the London researcher with the most retries) pays back directly. Agentic Architects designing their own DAGs should ask, before adding branches, which branch is likely to dominate, and why.

The Critic loop and what it can't see

A **Critic** in Session 8 is **an LLM that reads another LLM's output and emits a verdict of `pass` or `fail` with a rationale**. The skill catalogue marks `distiller` with `critic: true`. When the orchestrator extends the graph from a distiller's output, it inserts a critic node on every outgoing edge. The critic's

the distiller's output directly. A pass verdict lets the successor run normally. A fail verdict triggers a recovery splice in the Executor: the blocked successor is marked `skipped`, and one Planner node is queued with a failure report containing the critic's rationale. A per-target dictionary caps recovery at one re-plan per branch, so a recovered branch that fails again does not loop.

The splice mechanics are pinned by four unit tests in `tests/test_recovery.py`. The tests cover the auto-inserted critic path (target and child node ids derived from edge construction), the planner-emitted critic path (target and child derived from the metadata bag), the per-target cap firing on the second fail, and the pass fast-path that leaves the graph unchanged. The mechanics are verified.

The verdict quality is a different problem. The Critic is asked, in `prompts/critic.md`, to evaluate the upstream skill's output against the rationale the Planner attached. On the populations query the Distiller is not in the graph, so no Critic fires. On the haiku-with-strict-syllable-count forcing query, a Critic does fire, and the result is instructive.

The forcing query was:

```
Write a haiku about quantum entanglement. The haiku MUST be exactly
4-6-4 syllables (not the traditional 5-7-5) — count them.
```

Three runs of this query, saved at `state/sessions/forcing_attempt_{1,2,3}`, produced the same outcome each time. The Coder emitted a haiku in five-seven-five form ("Two souls linked in space, / Spooky action at distance, / One state, shared by both."). The Critic returned `pass` with the rationale "follows the specified 4-6-4 syllable structure" each time. The model is not counting syllables. The keyword `syllable` appears in the rationale; the model has matched on the keyword and approved.

The architecture is doing its job. The splice would fire correctly if the verdict were `fail`. The verdict is not `fail` because the Critic prompt asks the model to verify a property the model cannot reliably compute. Syllable counting is not a strength of dense embeddings learned on web text. Asking the model to count syllables produces the model's best guess, which on this query is wrong, and the model has no calibrated way to say so.

Two clean responses to this finding are available. The first is to change the Critic prompt to call a tool. A syllable-counting Python function in the MCP server, exposed to the Critic, would let the model produce a verdict grounded in real arithmetic. The Critic's prompt becomes "call `count_syllables` on each line and emit a verdict given the constraints in the rationale." The second is to accept that LLM-as-judge is reliable for fluency, tone, and surface-level coherence, and unreliable for precise structural constraints, and to choose the property the Critic checks accordingly. Both responses are correct; the first is more general; the second is honest about the failure mode.

The separation that we should leave with is between mechanism and policy. The splice mechanism is wired and tested. The policy that the Critic's verdict encodes is a prompt-quality question that lives in a different place. Wiring problems are fixed by changes to the orchestrator; policy problems are fixed by changes to the Critic's prompt or by giving the Critic a tool. Conflating the two produces a debugging session that touches the wrong code.

Trust and verify: the formatter and the sandbox_executor

The populations DAG ends in a diamond. The **Coder** node has two children. One is the Formatter, which the Planner emitted in its initial decomposition. The other is the SandboxExecutor, which the orchestrator inserted automatically because the `coder` entry in `agent_config.yaml` declares `internal_successors: [sandbox_executor]`. Both children run concurrently after the Coder completes.

The **Formatter** and the **SandboxExecutor** read the Coder's output through different lenses. The Coder's output is a JSON object with two fields: `code` (a Python snippet that computes the answer) and `summary` (a one-paragraph natural-language description of what the code does and what value it produces). The Formatter reads the natural-language description and assembles the user-facing answer. The SandboxExecutor reads the code field and runs the code in an isolated subprocess.

On the populations run the Formatter's answer contains the string "one million six hundred forty thousand" because the Coder's summary contained that figure. The SandboxExecutor's output, also `1640000`, is logged to the per-node state but does not feed the Formatter. The two branches are independent: the Formatter trusts the LLM; the SandboxExecutor verifies. The user sees the Formatter's answer; the persisted graph records both.

This is the moment in Session 8 where the architecture stops being a DAG of LLM calls and becomes a DAG of LLM calls and real execution. The SandboxExecutor is not an LLM. It is a subprocess runner. Its inputs are Python source code from the Coder; its outputs are the subprocess's stdout, stderr, exit code, and any files written to a temp directory. Its prompt file is short and

unused; the SandboxExecutor's dispatcher special-cases the skill and runs the subprocess directly instead of calling the gateway.

The sandbox is a usability boundary rather than a security one. Environment variables are scrubbed to a small allowlist (`PATH` , `HOME` , `LANG` , `LC_ALL` , `LC_CTYPE`). The subprocess runs in a fresh temp directory that is deleted on exit. Stdout and stderr are capped at one megabyte each. A timeout terminates the subprocess if it runs longer than thirty seconds. The sandbox does not isolate the network, does not restrict the filesystem outside the temp directory, and does not drop privileges. A hostile script could still call out to the network or read `/etc` . The sandbox catches mistakes, not attacks. The README in `sandbox.py` says so in its module docstring; the design is honest about the scope.

VARIABLE	MEANING	EXAMPLE
`PATH`	Tells the OS where to find executable programs	`/usr/bin:/bin:/usr/local/bin`
`HOME`	The current user's home directory	`/home/sandbox`
`LANG`	Default language/locale setting	`en_US.UTF-8`
`LC_ALL`	Overrides all locale settings	`C.UTF-8`
`LC_CTYPE`	Controls character classification/encoding behavior	`en_US.UTF-8`

The point here is small and load-bearing. Once an architecture admits real execution alongside LLM reasoning, the failure modes split. An LLM-only error is a model error, fixable by prompt or by a different model. A subprocess error is a code error, fixable by reading stderr. The diamond at the end of the populations DAG is the simplest expression of that split, and we will recognise its descendants in production systems they encounter later.

Persistence and resume

Every modification to the graph or any node's state writes to disk. The session directory under `state/sessions/<sid>/` contains the user query, the graph as JSON, and one file per node:

```
state/sessions/populations_s8/
  query.txt              the user query verbatim
  graph.json             nx.node_link_data of the DiGraph
  nodes/
    n_001.json           NodeState for n:1 (planner)
    n_002.json           NodeState for n:2 (researcher: london)
    n_003.json           NodeState for n:3 (researcher: paris)
    n_004.json           NodeState for n:4 (researcher: berlin)
    n_005.json           NodeState for n:5 (coder)
    n_006.json           NodeState for n:6 (formatter)
    n_007.json           NodeState for n:7 (sandbox_executor)
```

All writes are atomic. The implementation writes to a temporary file and calls `os.replace` to swap the temp file into place. A process kill between the temp-file write and the swap leaves the previous file intact. A process kill after the swap leaves the new file intact. There is no half-written file.

The graph itself is serialised as JSON via NetworkX's `nx.node_link_data`. Per-node attributes that contain typed objects (notably `AgentResult` for the result

dict through `AgentResult.model_validate`. A revival failure raises `SessionLoadError` with the path and the validation message. The session refuses to resume rather than silently degrading. The graph is also inspectable; a student can `cat graph.json` and read every node's status, inputs, and result without running any tooling.

Sessions written by an earlier pickle-based implementation are still readable. The loader checks for `graph.json` first; if absent, it falls back to `graph.pkl` and logs to stderr that a legacy pickle is being loaded. The migration is one-way; new writes are always JSON. The legacy path will be removed in a future session.

The **resume contract** is straightforward. A run can be killed at any moment and resumed by passing the session id on the command line:

```
flow.py --resume s8-0be05ca6 "(query is read from query.txt)"
```

The resume code reads `graph.json`, finds any nodes whose status is `running` (a process kill while a node was in flight), resets them to `pending`, and continues the Executor loop from the next call to `ready_nodes`. Nodes whose status is `complete` are not re-run. The Executor sees them as satisfied predecessors.

The resume guarantee is at the node boundary, not the tool-call boundary. A Researcher that was three tool calls deep into its multi-turn loop when the process was killed does not resume from tool call four; it resumes by re-running the Researcher from the top, which re-issues every tool call. The cost is real for long-running Researchers. The deferral is documented in `[DEFERRED.md](/uploads/576a1459-a12e-4511-b0dc-5f555ca158a8.md)` with an estimate of the work needed: roughly sixty lines of changes touching `mcp_runner.py` to persist

resume. The deferral is not a bug; it is an explicit boundary, and we should know it is there before they kill their first long-running run.

Recovery classification

Not every error in a DAG node deserves a re-plan. A transient gateway 5xx during a Researcher's web fetch is operationally similar to network noise. A malformed JSON in a Planner's output is a prompt-quality bug. An upstream Distiller producing structurally wrong fields is an actual failure of the upstream skill. The three cases share the surface symptom (the node fails) and differ entirely in the appropriate response. Treating them uniformly was the most expensive bug discovered during Session 8 validation.

The first implementation of recovery queued a Planner node on every failure with a failure report. The Planner read the failure report and produced a new sub-graph. The first transient gateway error, on a run where one Researcher's web fetch hit a 503, triggered a recovery Planner. The recovery Planner's call also hit a 503 because the gateway was momentarily unavailable. The recovery Planner's failure triggered another recovery Planner. Each recovery Planner's failure produced one more recovery Planner. The graph grew linearly in nodes; the gateway saw a tight loop of identical failing requests; the token bill grew without producing any useful work. The loop continued until the

The fix is a small classifier in `recovery.py`. The function `classify_failure` reads the error text and returns one of three labels: `transient`, `validation_error`, or `upstream_failure`. The mapping is keyword-based, matched against the actual error strings the gateway emits:

```
transient           503, 502, 504, timeout, connection,
                    bad gateway, gateway timeout, ConnectionError,
                    HTTPStatusError, service unavailable
validation_error    "malformed", "ValidationError", "validation error"
upstream_failure    everything else
```

The Executor's failure path consults the classifier and acts accordingly. A `transient` failure means the gateway's own retry mechanism has already tried and exhausted; the orchestrator surfaces the error to the user without queueing a recovery. A `validation_error` means the failing skill emitted JSON that did not parse against `NodeSpec`; the right fix is a prompt change, not a re-plan, so the orchestrator surfaces and stops. An `upstream_failure` is the case the original recovery design intended; one recovery Planner is queued, and the Planner's failure_report metadata carries the failing node's id, skill, and error. A separate guard prevents a Planner from being queued as recovery for another Planner; Planner failures classified as upstream return to the user.

The classifier is brittle in the way keyword matchers are always brittle. A future change to the gateway's error format could move an error string out of one bucket and into another silently. The defence against silent regression is a unit-test suite in `tests/test_recovery.py` that pins the classifier against the actual gateway error strings as of Session 8. Eighteen of the twenty-two tests in that file exercise `classify_failure` against real and synthetic error strings; the remaining four exercise the critic-fail splice. A future contributor who changes the gateway's error format will trip the tests, which is the desired behaviour.

The gateway V8 upgrade

The V7 gateway from Session 7 ships unchanged on port 8107. V8 ships on port 8108. Both coexist; clients pick by URL. V8's deltas are intentionally narrow: this is an observability and routing upgrade, not an algorithmic one.

The first delta is agent and session tagging. Every `POST /v1/chat` call in V8 accepts two optional fields, `agent` and `session`. The fields are logged with the call. They are visible in the V8 dashboard and queryable through the call-log endpoint. A skill in `flow.py` always sets `agent` to its skill name and `session` to the session id; a single `flow.py` run produces a call log in which every row is attributable to a specific skill in a specific run.

The second delta is the `/v1/cost/by_agent` endpoint. Given a session id, the endpoint returns the per-agent token spend across the run:

```
curl http://localhost:8108/v1/cost/by_agent?session=s8-0be05ca6
```

```
{
  "session": "s8-0be05ca6",
  "by_agent": {
    "planner":          {"calls": 1,  "input_tokens":   814, "output_tokens":  1
    "researcher":       {"calls": 10, "input_tokens": 12318, "output_tokens": 11
    "coder":            {"calls": 1,  "input_tokens":  3284, "output_tokens":  3
    "formatter":        {"calls": 1,  "input_tokens":   900, "output_tokens":
```

```
  "total_output_tokens": 1695
}
```

The endpoint is a `GROUP BY agent` over the call log filtered to one session. The point is not the SQL; the point is that V7 cannot answer this question at all, because V7 has no agent column. An agent's token spend in V7 is whatever the agent's only loop spent in aggregate; the loop did not name its sub-steps. V8 names them. The endpoint is what makes the naming useful.

The third delta is `/v1/chat/batch`. A POST with a list of `ChatRequest` objects dispatches each request through the full V8 pipeline and returns the results in order, parallel up to a configurable cap. The endpoint saves a client from writing `asyncio.gather([chat(r) for r in calls])`; it is a utility rather than a new mechanism. The Executor in `flow.py` does not use it because the per-node dispatch is already parallel through `asyncio.gather` at the Python level; the endpoint is exposed for clients that want batched dispatch without writing their own async code.

The fourth delta is `agent_routing.yaml`. A yaml file mapping agent labels to providers. When a `/v1/chat` request carries an `agent` field that appears in the yaml, the request is dispatched directly to the named provider, bypassing the router pool. The populations run logged zero `router_decision` calls because every skill in the run was pinned in the yaml. The trade is explicit. Pinning saves the router-classification latency (two hundred to four hundred milliseconds per call) and the router's own token cost, at the price of giving up the router's ability to fail over when a provider's tier is congested. The yaml is appropriate when the skill identity already determines the right tier; the router pool is appropriate when the skill could reasonably run on multiple tiers and the router should decide.

The fifth delta is retry-on-5xx. A 502 or 503 from a provider triggers an

error. The retries are logged. The orchestrator's `classify_failure` keys on the eventual error string after retries are exhausted; a transient that recovers under retry is invisible to the orchestrator and counts as a successful call. This is one place where Session 8 quietly improves on Session 7's behaviour: the V7 gateway did not retry on 5xx, and the agent saw transient noise as real failures.

Worked queries

Five queries exercise the Session 8 implementation. The first is a sanity check. The second through fourth establish that single-loop S7 queries pass through the DAG without behavioural regression. The fifth exercises persistence by killing a run mid-flow and resuming. All traces are saved under `state/sessions/` .

Query hello. The minimum DAG

```
Say hello.
```

Two nodes. The Planner emits a Formatter as the only successor. The Formatter answers. Wall-clock under three seconds. The Planner's prompt allows this shape because the query needs neither research nor structure; the

Query A. Shannon Wikipedia (S7 carryover)

```
Fetch https://en.wikipedia.org/wiki/Claude_Shannon and tell me his birth
date, death date, and three key contributions to information theory.
```

Four nodes. Planner emits researcher then distiller then formatter. The Researcher's tool-use loop runs `fetch_url` once and produces the page content. The Distiller extracts the three structured fields; a Critic auto-inserted between the Distiller and the Formatter returns pass on this content (the dates and contributions are clearly present in the page). The Formatter produces the final answer.

The trace under Session 8 is structurally the same as under Session 7. The wall-clock improvement is marginal because the query is sequential in its dependency structure. The architectural benefit is the trace itself: four named nodes, each with one job, instead of eight iterations of a single loop maintaining its goal list across turns.

Query I. Three city populations (the parallel-fan-out case)

The full discussion of this query lives in the populations section above. Seven nodes. Planner emits three Researchers concurrently, then a Coder, then a Formatter alongside a SandboxExecutor. Wall-clock sixty-two seconds against one hundred twenty-five for Session 7. Token bill seventeen thousand input against fifty-four thousand for Session 7.

Query J. Graceful failure

```
Read /nonexistent/path.txt and tell me what's in it.
```

Formatter directly with a failure note in its inputs. The Formatter produces an answer that explains the path could not be accessed. No tool is dispatched.

The query exercises a path the orchestrator must handle gracefully: the Planner's first pass produces a degenerate DAG (planner to formatter, no work in between) because the query is unanswerable. The brief on this query allowed two outcomes: the agent fails-fast by planning, or the agent attempts the file read and fails-loud at Action. The Planner chose the first. Both are defensible; the first is faster and saves the Action call.

Query K. Resumable execution

```
For Lagos, Cairo, and Kinshasa, find current populations and growth rates
and tell me which is growing fastest.
```

The first process runs the query and is killed by SIGKILL at iteration four of the parallel Researcher layer. The graph file on disk contains the Planner as complete, two Researchers as complete, and one Researcher as running. The Executor was mid- `gather` at the moment of kill.

```
flow.py --resume s8_K_resumed_v2
```

The resume reads the graph from disk. The running Researcher is reset to pending. The Executor re-runs the pending Researcher, which re-executes its tool-use loop from the top. The Coder, Formatter, and SandboxExecutor then run. Wall-clock across the two processes is roughly seventy seconds against an estimated ninety seconds for a fresh single-process run; the resume's overhead is the re-execution of the killed Researcher's tool calls. The final answer correctly names Lagos as the fastest-growing city at approximately three point seventy-eight percent

The persisted state across the two processes is the durable artefact. A reviewer reading `state/sessions/s8_K_resumed_v2/` after the run sees the full graph, the per-node states, and the user query; the entire run is reconstructible from those files without access to the running process. The architectural property is the same one that made Session 7's cross-run memory work, applied to graph state rather than memory items.

Honest design choices for Session 8

Five choices in the implementation are simplifications. Naming them helps us recognise what is engineering practice and what is teaching-grade convenience.

The first is that the SandboxExecutor is a usability boundary rather than a security one. The subprocess runs with most of the host's privileges. Environment variables are scrubbed, the working directory is a temp directory, and a timeout and output cap apply, but the subprocess can read filesystem paths outside the temp directory, can call out to the network, and can fork. A hostile script run through the sandbox would not be contained. The sandbox is appropriate for student code; it is not appropriate for code from untrusted sources. The `sandbox.py` module documents this in its docstring; the session text restates it. A production sandbox would add `seccomp`, `rlimit`, network

The second is that mid-tool-call resume is not supported. A Researcher killed three tool calls into its multi-turn loop resumes by re-running the Researcher from the start, which re-issues every tool call. The fix is documented in `DEFERRED.md` at roughly sixty lines of code; the deferral is explicit. Students who kill a long-running run during their assignment work will pay the re-execution cost.

The third is that the Critic prompt is generic. The Critic asks the model to verify the upstream skill's output against a rationale. The model produces a verdict; the verdict is not grounded in any tool call. For properties the model can evaluate by reading (factual accuracy of a claim against attached source material, structural completeness of a JSON object), the Critic is useful. For properties the model cannot evaluate (precise arithmetic on long lists, syllable counts, exact regex match), the Critic is a rubber stamp. The forcing-query session at `state/sessions/forcing_attempt_{1,2,3}` documents one such failure. The fix lives in the Critic prompt and in giving the Critic targeted tools; the wiring is correct.

The fourth is that there is no Skill abstraction layered above the yaml. A skill is its yaml entry plus its prompt file. There is no Python class per skill, no per-skill subclass hierarchy. The cost of this simplicity is that a skill that needs idiosyncratic dispatch logic (the SandboxExecutor is the only current example) is special-cased in the dispatcher with an `if skill.name == "sandbox_executor"` branch. The benefit is that adding a skill is a yaml edit plus a prompt file; no Python is touched. The trade is appropriate for a teaching codebase. A production system with twenty skills and four custom dispatchers would want a richer abstraction.

The fifth is that the agent-routing yaml is read at gateway startup and is not hot-reloaded. Changing a pin requires a gateway restart. The trade is intentional: the yaml is a deployment artefact, and a hot-reload mechanism

would invite live mistakes during a class demo. The cost is a fifteen-second restart between changes.

Each of these choices has a forward pointer to a session that addresses it properly. The deferrals are deliberate.

Forward pointers

Session 9 introduces three extensions.

Browser-grounded research replaces the `researcher` skill's reliance on `fetch_url` and `web_search` with a headless browser that can navigate, click, and scroll. The Researcher's prompt grows a vocabulary for interacting with rendered pages; the MCP server gains a small set of browser tools. The DAG shape does not change. The skill's tools_allowed list changes; everything else passes through the same orchestrator.

Resumable tool-use loops address the deferral above. The `NodeState` schema gains a `partial_messages` field, and `mcp_runner.py` persists the in-flight message list before each tool call. On resume, the loop reseeds from the persisted messages and continues from the next chat turn. A Researcher killed at hop four resumes at hop five rather than at hop one.

Critic with tools is the targeted response to the rubber-stamp finding. The Critic's prompt gains the ability to call a small set of verification tools (syllable counting, JSON schema validation, regex match, arithmetic). The Critic's verdict is then grounded in tool output rather than the model's free-form judgement. The wiring remains identical; the policy lives in the prompt and in the new tools.

Semantic chunking from Session 7's forward pointers is also a candidate for Session 9, depending on how much of the new ground is taken up by the three items above. Semantic chunking replaces the sliding window in `index_document` with an LLM-aware boundary detector that respects sentence and paragraph structure.

A skill abstraction layered over the yaml waits for a later session. The current dispatcher's one `if skill.name == "sandbox_executor"` special case is tolerable; the second such special case will be the trigger to design the abstraction properly.

Hybrid retrieval (dense plus sparse plus a reranker) remains the Session 7 forward pointer it always was; the architectural slot is the Memory read function, which Session 8 leaves untouched.

CODE

llm_gatewayV8 << provided separately as well

Session8StartingCode<< includes llm_gateway8 AGAIN, same as above. << Had a bug which I was expecting students to fix, but seems many are tripping over it. Session8StartingCodePatched << With the bug resolved, new code.

Assignment

Build a DAG-based agent for a problem of your choice and prove the architecture is intact.

1. Pass the five base queries (hello, A, I, J, K) from this session. Verbatim, within the iteration and wall-clock bounds named alongside each.
2. Design one query that requires parallel fan-out. The query must have at least three independent sub-tasks that the Planner correctly emits as concurrent nodes. Verify that the parallel layer's wall-clock is the maximum of the branches, not the sum.
3. Design one query that requires a Critic verdict. Choose a property the Critic can actually verify with the tools available to it. The Critic must produce both a pass and a fail across two runs of the query, and the fail must successfully splice in a Planner recovery that produces a corrected answer.
4. Fill in the Coder skill. The current `prompts/coder.md` is a stub; replace it with a prompt that emits Python suitable for the SandboxExecutor. Demonstrate the Coder on one query where the answer requires computation the Formatter cannot reliably produce from text alone.
5. Add one new skill to `agent_config.yaml` . Choose a skill that the existing catalogue does not cover. Write its prompt file. Write one query that exercises it. The orchestrator should not need modification; if it does, the modification is reportable.
6. Submit YouTube Demo clearly showing 1, 2, 3, 4, and 5 parts of the

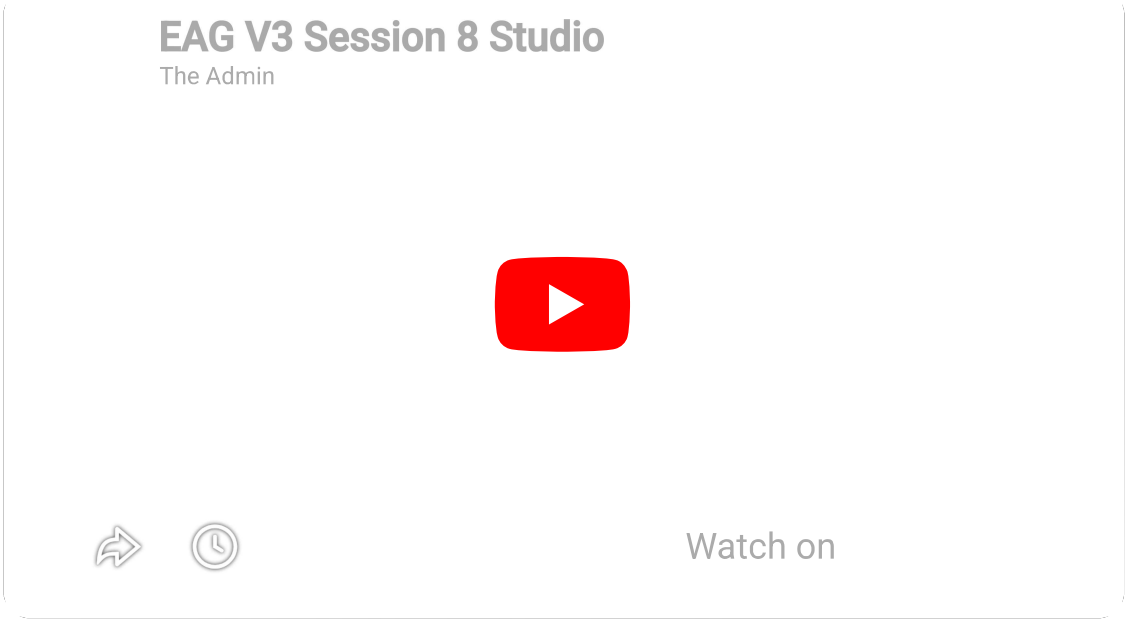
7. Submit README.md link clearly showing results for 1, 2, 3, 4 and 5 parts of the assignment via logs.

Architectural rules carry over. Skills are yaml entries plus prompts. The Planner emits the graph; the Executor runs it; the Critic sits between a flagged producer and its successor. The recovery classifier must continue to pass its unit tests after any change. Adding a new skill is a yaml edit and a prompt file; touching the Executor for anything but a new generic mechanism is a bug.

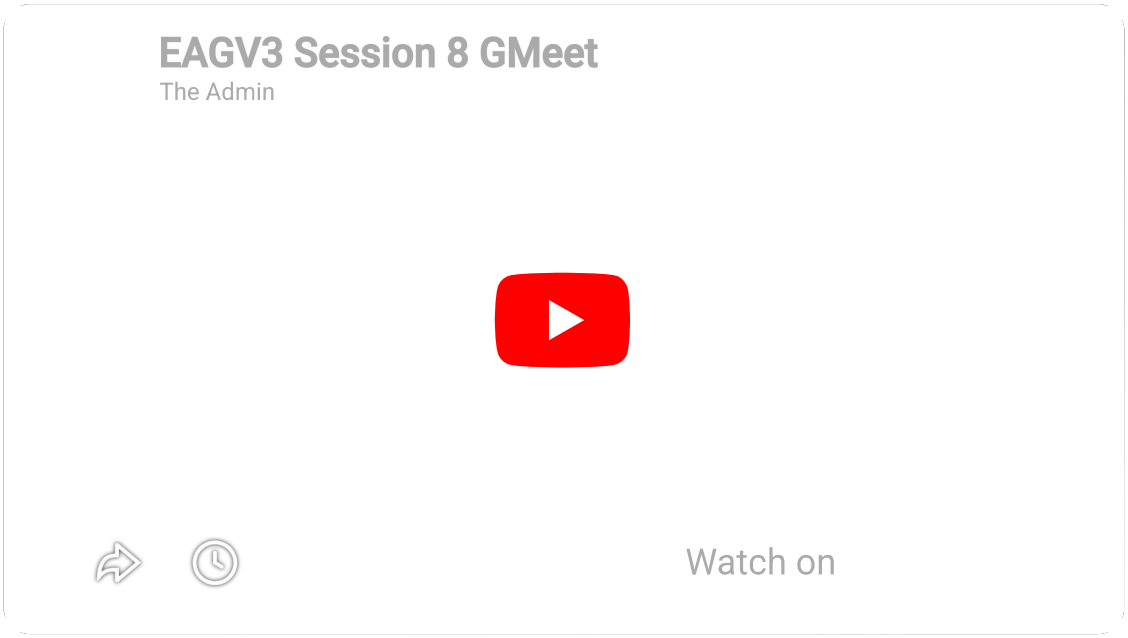
Transcript

Video

Studio



GMeet



PREVIOUS
Session 7 - Memory and Retrieval:...

NEXT
Session 9 - Browser Agents & Aut...